ЛАБОРАТОРНАЯ РАБОТА № 4

Дисциплина: AI Agents for Enterprise Processes Automation

Тема: Разработка LLM-агента на основе паттерна ReAct

Выполнила: Гулум Манасова

Университет: Ala-Too International University

1. Цель работы

Цель лабораторной работы — изучить принцип работы LLM-агентов на основе паттерна ReAct (Reasoning and Acting), реализовать агента с использованием языковой модели и инструментов, протестировать его работу и оценить результаты с помощью журнала выполнения и метрик.

2. Что было сделано

В ходе лабораторной работы был разработан ReAct-агент на Python, использующий локальную языковую модель Qwen через Ollama.

В проекте реализованы следующие компоненты:

- **agent.py** — основной цикл агента, обработка ответов модели, вызов инструментов и формирование итогового ответа.
- **tools.py** — инструменты поиска по внутренним документам и выполнения математических вычислений.
- **tests.py** — офлайн-тестирование поведения агента и тестирование с реальной языковой моделью.
- **analyze.py** — анализ журнала выполнения и расчёт основных метрик.
- **docs/** — внутренние документы компании, используемые инструментом поиска.
- **agent_log.jsonl** — журнал событий, записываемый во время работы агента.

Для запуска языковой модели использовалась Ollama с моделью Qwen. Это позволило выполнять запросы локально, без использования облачного API сторонней языковой модели.

2.1. Реализованные инструменты

1. text_search — поиск по документам.

Инструмент выполняет поиск информации во внутренних текстовых документах. Он используется для получения ответов на вопросы о правилах компании, отпуске, удалённой работе, командировках и компенсациях.

2. calculator — калькулятор.

Инструмент выполняет математические вычисления по переданному выражению. Для повышения безопасности вычисления обрабатываются с использованием AST, а не небезопасного вызова `eval()`.

2.2. Принцип работы ReAct

ReAct объединяет рассуждение языковой модели и выполнение действий с помощью инструментов.

Цикл работы агента:

1. Пользователь передаёт вопрос.
2. Модель анализирует задачу и выбирает следующий шаг.
3. Если требуется инструмент, модель формирует действие `Action` и входные данные `Action Input`.
4. Агент выполняет выбранный инструмент.
5. Результат инструмента возвращается модели в виде `Observation`.
6. Цикл повторяется до получения `Final Answer` либо достижения максимального количества шагов.

Если модель выдаёт ответ в неправильном формате, агент формирует сообщение об ошибке. При попытке вызвать неизвестный инструмент возвращается соответствующее сообщение. Для предотвращения бесконечного цикла используется ограничение количества шагов.

Схема цикла:

Вопрос пользователя → Thought → Action → Выполнение инструмента → Observation → Следующий шаг или Final Answer.

3. Результаты тестирования

Для проверки работы агента были выполнены пять тестовых заданий с использованием локальной языковой модели.

№	Тип задачи	Ожидаемый результат	Полученный результат	Итог
1	Поиск информации об отпуске	28 календарных дней	28 календарных дней	Успешно
2	Поиск информации об удалённой работе	2 дня в неделю	2 дня в неделю	Успешно
3	Математическое вычисление	31 800	31 800	Успешно

4	Расчёт суточных за 6 дней командировки	9 000 сом	Ответ не получен в пределах лимита шагов	Неуспешно
5	Расчёт компенсации за 250 км	3 000 сом	3 000 сом	Успешно

Точность оценивалась сравнением ответа агента с ожидаемым результатом. Четыре из пяти заданий завершились успешно.

3.1. Метрики работы агента

По журналу выполнения `agent_log.jsonl` были рассчитаны следующие показатели для пяти основных тестовых запусков:

Метрика	Результат
Количество тестов	5
Количество правильных ответов	4
Точность ответов	80%
Среднее количество шагов на вопрос	10
Средняя задержка языковой модели на шаг	63,52 секунды
Суммарное количество токенов	35 859
Успешность распознанных вызовов инструментов	100%

Показатель успешности вызовов инструментов относится только к распознанным вызовам, для которых была определена операция. Он не отражает общую точность агента и не учитывает все ошибки формата и незавершённые запуски.

Средняя задержка измеряет время работы языковой модели на одном шаге, а не полное время решения одного вопроса. Значение зависит от локального оборудования, выбранной модели и длины ответов.

4. Проверка обработки ошибок

Помимо пяти основных заданий, было выполнено офлайн-тестирование с заранее заданными ответами модели. Это позволило проверить отдельные сценарии независимо от случайных изменений поведения языковой модели.

Были проверены следующие ситуации:

- вызов инструмента поиска `text_search`;

- попытка вызова неизвестного инструмента `weather`;
- выполнение вычисления с помощью `calculator`;
- получение ответа в неправильном формате;
- формирование итогового ответа `Final Answer`.

Все пять офлайн-проверок завершились успешно. При неправильном формате агент сформировал сообщение об ошибке, а при вызове неизвестного инструмента вернул соответствующее уведомление.

Дополнительно был проведён тест с запросом об использовании инструмента `weather` для получения прогноза погоды в Бишкеке. Агент сообщил, что такой инструмент недоступен, и перечислил имеющиеся инструменты.

5. Что не получилось

Ошибка 1. Превышение лимита шагов при расчёте суточных

При вопросе о размере суточных за шестидневную командировку агент не смог получить итоговый ответ в пределах установленного количества шагов.

Фрагмент результата:

```
Не удалось получить ответ за отведённое число шагов.
```

Возможная причина — затруднение при последовательном выполнении поиска и вычисления либо повторение отдельных действий без перехода к окончательному ответу.

Для дальнейшего улучшения можно добавить примеры правильных траекторий ReAct в промпт, уточнить правила использования калькулятора и настроить обработку ситуации, когда необходимая информация не найдена.

Ошибка 2. Недостаточно эффективная обработка отсутствующей информации

При запросе об информации, отсутствующей во внутренних документах, агент также не смог завершить работу в пределах лимита шагов.

Возможная причина — недостаточно чёткое правило завершения поиска, если релевантная информация отсутствует.

Для улучшения поведения можно установить явное правило: после нескольких неудачных поисков агент должен сообщить, что в предоставленных документах нет необходимой информации, и не должен придумывать ответ.

Эти случаи показывают, что ограничение числа шагов защищает систему от бесконечного выполнения, но для повышения надёжности необходимо также улучшить стратегию поиска и условия завершения работы.

6. Что усвоил

В ходе лабораторной работы я изучила принцип работы LLM-агентов на основе паттерна ReAct и научилась связывать языковую модель с внешними инструментами. Я реализовала поиск по внутренним документам и калькулятор, а также проверила обработку неправильного формата ответа и неизвестного инструмента. Я научилась анализировать журнал JSONL и рассчитывать метрики работы агента. Результаты тестирования показали, что даже при наличии работающих инструментов агент может зацикливаться или не завершать задачу, поэтому важны ограничение количества шагов и корректная обработка ошибок.

7. Использование искусственного интеллекта

При выполнении лабораторной работы использовались следующие ИИ-инструменты:

- **ChatGPT** — для объяснения принципа работы ReAct, помощи при разработке и отладке кода, анализа результатов тестирования и подготовки структуры отчёта.
- **Ollama с моделью Qwen** — для локального запуска языковой модели, обработки запросов пользователя, выбора инструментов и формирования итоговых ответов.

Результаты работы агента проверялись с помощью офлайн-тестов, пяти основных тестовых заданий и журнала выполнения. Использование ИИ-инструментов не заменяло проверку результатов: корректность ответов оценивалась сравнением с ожидаемыми значениями.

8. Вывод

В ходе лабораторной работы был разработан и протестирован локальный LLM-агент на основе паттерна ReAct. Агент использует инструменты поиска по внутренним документам и математических вычислений, а также записывает события выполнения в журнал JSONL.

В пяти основных тестах агент правильно выполнил четыре задания, достигнув точности 80%. Офлайн-тестирование подтвердило работу основных сценариев: вызова инструментов, обработки неизвестного инструмента, ошибки формата и формирования итогового ответа.

Основная выявленная проблема — невозможность завершить некоторые задачи в пределах установленного лимита шагов. В дальнейшем качество агента можно улучшить с помощью более точных инструкций, примеров правильных траекторий, улучшенной обработки отсутствующей информации и оптимизации количества обращений к языковой модели.

9. Приложения

К отчёту прилагаются скриншоты:

1. Структура проекта в PyCharm.
2. Код основного агента `agent.py`.
3. Реализация инструментов в `tools.py`.
4. Результат успешного запуска `python tests.py`.
5. Результаты пяти тестов с локальной моделью.
6. Результат выполнения `python analyze.py`.
7. Пример журнала `agent_log.jsonl`.
8. Пример обработки неизвестного инструмента `weather`.

Исходный код: ссылка на репозиторий GitHub или приложенный архив проекта.